Studio Nx

JS 70 — loadDashboard() (+ page-errors.js)

2026-10-12

JS 70 — `loadDashboard()` (+ `core/page-errors.js`)

항목	내용
파일	`studionx/frontend/static/js/pages/mypage.js`, 새 파일 `static/js/core/page-errors.js`
작업	P1-17 마이페이지 JS: 대시보드 데이터 받기 (PRD 01 FR-121)
작성	Claude (AI), 2026-10-12, 사용자 검토
이전 · 다음	69 `GET /me/dashboard` 라우터 · 71 (다음)

1. 하는 일

`GET /api/v1/me/dashboard`를 불러서 마이페이지에 필요한 데이터를 한 번에 받아 옵니다. 받지 못하면 실패한 이유에 맞는 화면으로 옮겨 가고 `null`을 돌려줍니다.

상황	결과
성공	`{ user, storage, ai, ready_projects, recent_projects }` 객체를 돌려줍니다
401 (로그인이 풀림)	`home.html#login?next=%2FmyPage.html`로 이동하고 `null`을 돌려줍니다. 로그인하면 마이페이지로 돌아옵니다
400, 403, 404, 429, 503, 504	같은 번호의 오류 화면 `errors/<번호>.html`
그 밖의 500 이상 (예: 502, 422)	`errors/500.html`
서버에 닿지 못함 (네트워크 오류)	`errors/503.html`

2. 언제 쓰나

호출하는 곳	화면
페이지 시작 (`mypage.js` 맨 아래, 이후 단계에서 연결)	`const data = await loadDashboard(); if (!data) return;` 다음에 렌더 함수들(71~75)에 값을 나눠 줍니다

실패했을 때의 이동 규칙은 userinfo(`loadMe`)와 myProject(`loadProjects`)가 쓰는 규칙과 같습니다 (2026-10-06 결정: 본 데이터 API 오류는 오류 화면으로). 세 번째 페이지가 되었으므로 이번에 **공통 파일 `core/page-errors.js`**로 따로 뺐고, 이 파일은 지금 `mypage.js`에서만 씁니다. userinfo.js와 myproject.js가 같은 규칙을 갖고 있는 것은 그대로 두었습니다. 필요하면 나중에 두 파일도 이 함수를 쓰도록 바꿀 수 있습니다.

3. 코드

import (`mypage.js` 맨 위, 이번에 추가한 것만)

```
import { api } from '../core/api.js';
import { redirectOnLoadError } from '../core/page-errors.js';
```

함수 (`mypage.js`)

```
export async function loadDashboard() {
  try {
    return await api.get('/me/dashboard');
  } catch (err) {
    redirectOnLoadError(err);
    return null;
  }
}
```

`core/page-errors.js` (새 파일)

```
import { loginUrl } from './auth.js';

const ERROR_PAGES = [400, 403, 404, 429, 500, 503, 504];

export function redirectOnLoadError(err) {
  if (err.status === 401) {
    location.replace(loginUrl(location.pathname + location.search + location.hash));
    return;
  }
  let page = 500;
  if (ERROR_PAGES.indexOf(err.status) >= 0) page = err.status;
  else if (err.status === 0) page = 503;
  location.replace('errors/' + page + '.html');
}
```

최신 문법으로 쓰면 (공부용이며, 실제 코드는 위에 있습니다)

```
export const redirectOnLoadError = ({ status }) => {
  if (status === 401) return location.replace(loginUrl(location.pathname + location.search + location.ha
sh));
  const page = ERROR_PAGES.includes(status) ? status : status === 0 ? 503 : 500;
  location.replace(`errors/${page}.html`);
};
```

최신 문법	실제 코드	뜻
`({ status }) =>`	`function (err)`와 `err.status`	매개변수에서 바로 칸을 꺼냅니다 (구조 분해)
`ERROR_PAGES.includes(status)`	`ERROR_PAGES.indexOf(err.status) >= 0`	목록에 있는지 확인합니다
`` `errors/${page}.html` ``	`'errors/' + page + '.html'`	문자열에 값을 끼워 넣습니다
삼항 연산자 이어 쓰기	`let page = 500; if … else if …`	여러 갈래로 나눕니다. 이어 쓰면 읽기 어렵습니다

문법 풀이

import 문법	뜻
`import { api } from '../core/api.js'`	토큰을 붙여 서버 API를 부릅니다. 만료되면 한 번 재발급한 뒤 다시 보내고, 실패하면 `ApiError`를 던집니다 (P1-02)
`import { redirectOnLoadError } from '../core/page-errors.js'`	이번에 만든 공통 함수입니다

식	뜻
`try { return await … } catch (err) { … }`	성공하면 응답을 돌려주고, 실패하면 `catch`로 갑니다. `return await`이라서 실패도 이 `try` 안에서 잡힙니다
`err.status`	`ApiError`의 HTTP 상태 번호입니다. 서버에 닿지 못했으면 0입니다
`location.replace(주소)`	그 주소로 옮겨 가면서, 방문 기록에 지금 페이지를 남기지 않습니다. 뒤로 가기로 깨진 화면에 돌아오지 않게 하려는 것입니다
`loginUrl(경로)`	`home.html#login?next=…` 주소를 만듭니다 (P1-02 04). 경로, 쿼리, 해시를 모두 실어 보내서 로그인 뒤에 같은 화면으로 돌아옵니다

4. 흐름

1. `GET /me/dashboard`를 보냅니다 (토큰이 없거나 만료되었으면 `api.js`가 먼저 재발급을 시도합니다).
2. 성공하면 응답 객체를 그대로 돌려줍니다.
3. 실패하면 `redirectOnLoadError`가 상태 번호에 맞는 화면으로 옮겨 가고, `null`을 돌려줍니다.

5. 왜 이렇게

선택	이유
실패하면 `null`을 돌려줌	부르는 쪽이 `if (!data) return;` 한 줄로 멈출 수 있습니다. 다른 화면으로 옮겨 가는 중에 그리기를 하지 않게 합니다
이동 규칙을 `core/page-errors.js`로 뺌	세 번째 페이지에서 같은 규칙이 필요해졌습니다 (JS 30의 메모). 규칙이 한 곳에만 있으면 나중에 바꿀 때 한 번만 고치면 됩니다
userinfo.js, myproject.js는 그대로 둠	이미 검토와 시험이 끝난 코드를 이번 작업에서 건드리지 않기 위해서입니다
401은 오류 화면이 아니라 로그인으로	"다시 로그인"이 맞는 응답입니다 (2026-10-06 결정)
이동 주소에 `location.hash`까지 실음	로그인한 뒤에 같은 화면 상태로 돌아오게 합니다

6. 시험 (실제 myPage.html, 실제 서버, 헤드리스 Chrome, 23개)

시험 계정 `sitetest.p117@example.com`으로 가입하고 로그인한 뒤, 프로젝트 3개를 만들고 페이지 안에서 함수를 불렀습니다. 실패는 `window.fetch`를 가짜로 바꿔서 서버 응답을 흉내 냈고, 이동은 브라우저의 Navigation API로 막은 채 목적지만 읽었습니다.

#	상황	결과
01	정상 요청	다섯 칸(`user`, `storage`, `ai`, `ready_projects`, `recent_projects`), 닉네임 `p117_user`, 최근 프로젝트 3개, 이어 하기 3개이고, 이동은 일어나지 않습니다
02~08	가짜 응답 500, 404, 403, 429, 503, 502, 422	모두 `null`을 돌려주고 `/errors/500.html`, `404`, `403`, `429`, `503`, `500`, `500`으로 이동합니다
09	네트워크 끊김	`null`을 돌려주고 `/errors/503.html`로 이동합니다
10	가짜 401	`null`을 돌려주고 `/home.html#login?next=%2FmyPage.html`로 이동합니다
11	진짜 로그아웃한 뒤 새로 열어서 호출	재발급도 실패해서 `null`을 돌려주고 같은 로그인 주소로 이동합니다

23개 모두 통과했습니다. 시험이 만든 계정과 프로젝트는 다음 함수 시험이 끝난 뒤에 함께 지웁니다 (서버와 계정은 계속 쓰고 있습니다).